GIT HANDS-ON LABS (HOL)

COMPREHENSIVE SOLUTIONS GUIDE (LABS 1–5)

Subject: Version Control using Git & Remote Repository Integration
Estimated Duration: 2 Hours | **Target Environment:** Windows / Git Bash

---

> **Important Notice regarding Security & Credentials:**
> As specified in the operational rules, always create a **free personal GitHub/GitLab account** for these lab exercises. **Do not use corporate or organizational credentials** (e.g., Cognizant credentials) to authenticate or log in to public cloud git repositories.

Lab 1: Git Configuration Machine Setup & Core Repository Operations

Objectives: Become familiar with foundational Git commands (`git init`, `git status`, `git add`, `git commit`, `git push`, `git pull`), integrate Notepad++ as the default global text editor, and successfully track and save your first source files.

Step 1.1: Verify Git Client Installation

Launch the Git Bash shell on your local machine and execute the following command to verify that the Git client is correctly installed and accessible via your path:

```
$ git version
# Expected Output Example:
git version 2.21.0.windows.1
```

Step 1.2: Configure Global User Profile Credentials

Set up your identity across all local repositories by establishing a global username and email address. Replace the values below with your personal GitLab/GitHub details:

```
$ git config --global user.name "username"
$ git config --global user.email "username@example.com"
```

To verify that your global configurations have been written correctly, query the global list:

```
$ git config --global --list
# Output:
user.name=username
user.email=username@example.com
```

Step 1.3: Integrate Notepad++ as the Default Git Editor

If Git Bash fails to resolve the `notepad++` command natively, it indicates that the binary is missing from your Windows Environment Path Variable. Add it via: *Control Panel → System → Advanced System Settings → Advanced Tab → Environment Variables → Edit User Variable "Path"* and append the installation path (e.g., `C:\Program Files\Notepad++`).

Once pathing is fixed, create a persistent profile alias and declare Notepad++ as the global text editor for commits and interactive operations:

```
$ notepad++ ~/.bash_profile
# Add the following line inside the opened file, save, and exit:
alias npp='notepad++.exe -multiInst -nosession'

$ source ~/.bash_profile
$ git config --global core.editor "notepad++.exe -multiInst -nosession"
```

Verify that the editor binding is functional by forcing Git to open the global config in edit mode (this should open a new, detached Notepad++ window instantly):

```
$ git config --global -e
# Hint: Waiting for your editor to close the file...
```

Step 1.4: Initialize the Local Git Repository

Initialize a new localized empty Git repository inside a project folder named `GitDemo`, change directory into it, and inspect hidden version tracking elements:

```
$ git init GitDemo
# Output: Initialized empty Git repository in D:/Development_Avecto/GitDemo/.git/

$ cd GitDemo
$ ls -al
# Output shows the hidden control directory: .git/
```

Step 1.5: Track, Stage, and Commit a File

Create a basic plaintext file titled `welcome.txt`, verify its presence, review the status of the workspace, stage it, and commit the modification:

```
$ echo "Welcome to the version control" >> welcome.txt
$ cat welcome.txt
Welcome to the version control

$ git status
# On branch master
# No commits yet
# Untracked files: (use "git add <file>..." to include in what will be committed)
#   welcome.txt

$ git add welcome.txt
$ git commit
# Notepad++ will automatically launch. Write your multi-line commit message, save and
close.
# Output example: [master (root-commit) a1b2c3d] Initial commit containing
welcome.txt

$ git status
# On branch master
# nothing to commit, working tree clean
```

Step 1.6: Connect and Synchronize with the Remote Repository

Establish a connection with your remote project created on GitLab/GitHub and sync changes:

```
$ git remote add origin https://gitlab.com/your-username/GitDemo.git
$ git pull origin master --allow-unrelated-histories
$ git push -u origin master
```

Lab 2: Declaring Exclusions with Git Ignore (.gitignore)

Objectives: Comprehend the usage of the `.gitignore` configuration pattern to enforce rules that prevent unnecessary metadata, runtime logs, binaries, or security artifacts from accidentally entering the version control lifecycle.

Step 2.1: Simulate System Generation of Unwanted Runtime Files

Generate a dedicated subdirectory named `log` alongside a random runtime tracking file with a `.log` extension inside your project workspace:

```
$ mkdir log
$ touch log/system_runtime.log
$ touch debug_execution.log
$ git status
# Output lists 'debug_execution.log' and 'log/' as untracked assets.
```

Step 2.2: Define the Exclusion Rules Matrix

Construct a tracking file named `.gitignore` directly at the root level of your project directory structure. Add specific rules targeting both specific file extensions and whole directories:

```
$ echo "*.log" >> .gitignore
$ echo "log/" >> .gitignore
$ cat .gitignore
*.log
log/
```

Step 2.3: Validate Rule Enforcement via Workspace Status

Execute the status validator tool to prove that the rule configurations match execution criteria. The log files and directories should disappear from the tracking dashboard, leaving only the `.gitignore` structure itself visible:

```
$ git status
# On branch master
# Untracked files:
#   .gitignore
# Note: debug_execution.log and log/ are properly masked and excluded!

$ git add .gitignore
$ git commit -m "Configure .gitignore matrix to permanently block .log files and
runtime folders"
$ git status
# nothing to commit, working tree clean
```

Lab 3: Git Branching and Merging Operations

Objectives: Construct isolated, parallel feature lines utilizing branches, modify contents independently, safely reintegrate the completed work vector back into the master trunk line, and clean up historical pointers.

Step 3.1: Initialize a Feature Branch

Create a separate, non-destructive working line named `GitNewBranch`, discover all locally available branch branches, and note the marker indicating the active focus context:

```
$ git branch GitNewBranch
$ git branch
#   GitNewBranch
# * master (the asterisk denotes the current working branch)
```

Step 3.2: Perform Isolated Work inside the Branch

Switch your workspace context to the new branch vector, insert workspace modifications, and lock them securely into the localized history block via commit:

```
$ git checkout GitNewBranch
# Switched to branch 'GitNewBranch'

$ echo "This content belongs specifically to the feature branch vector." >>
feature_delta.txt
$ git add feature_delta.txt
$ git commit -m "Add feature_delta.txt with target metrics within separate branch
context"
$ git status
# On branch GitNewBranch - nothing to commit, working tree clean
```

Step 3.3: Evaluate Differences Against the Master Trunk Line

Return context focusing back to the primary `master` trunk line, and query differences textually as well as visually via the external `P4Merge` wrapper configuration:

```
$ git checkout master
# Switched to branch 'master'

# CLI textual difference inspection:
$ git diff master GitNewBranch

# GUI visual validation engine:
$ git difftool master GitNewBranch
# Launching external tool comparison view using P4Merge...
```

Step 3.4: Reintegrate Work Vector and Analyze Topology

Execute a clean fast-forward or structural merge to consolidate the changes into the master trunk line. Then, view the visual logging topology map:

```
$ git merge GitNewBranch
# Updating a1b2c3d..e5f6g7h
# Fast-forward
#  feature_delta.txt | 1 +
#  1 file changed, 1 insertion(+)

$ git log --oneline --graph --decorate
# * e5f6g7h (HEAD -> master, origin/master, GitNewBranch) Add feature_delta.txt with
target metrics
# * a1b2c3d Configure .gitignore matrix to permanently block .log files and runtime
folders
```

Step 3.5: Post-Merge Linear Cleanup

Because the branch line context has been safely assimilated, purge the obsolete label and confirm the workspace status structure:

```
$ git branch -d GitNewBranch
# Deleted branch GitNewBranch (was e5f6g7h).

$ git status
# On branch master, working tree clean
```

Lab 4: Advanced Git Merge Conflict Resolution (3-Way Merge)

Objectives: Manage and resolve race conditions and textual conflicts that occur when two separate work paths modify the exact same structural components within a file before synchronization.

Step 4.1: Ensure Workspace Stability & Establish Branch Vector

Verify that your working tree is entirely clear, establish a new alternative execution branch named `GitWork`, transition your terminal workspace context into it, and generate a new structural file named `hello.xml`:

```
$ git status
# nothing to commit, working tree clean

$ git checkout -b GitWork
# Switched to a new branch 'GitWork'

$ echo "<root><data>Hello from the isolated GitWork branch path</data></root>" >
hello.xml
$ git status
$ git add hello.xml
$ git commit -m "Initialize hello.xml inside the alternative GitWork context"
```

Step 4.2: Introduce Conflicting Structural Modifications in Master

Return execution focus back to the primary `master` trunk. Modify the identical file line bounds by creating a conflicting `hello.xml` layout file locally, and commit these changes immediately:

```
$ git checkout master
# Switched to branch 'master'

$ echo "<root><data>Hello from the conflicting Master Trunk path</data></root>" >
hello.xml
$ git add hello.xml
$ git commit -m "Create a conflicting version of hello.xml directly on the master
track"
```

Step 4.3: Trace the Multi-Axis Tree Divergence Topology

Map out the cross-axis structural network split visually within the terminal history tracker to observe the path separation:

```
$ git log --oneline --graph --decorate --all
# * 9999999 (HEAD -> master) Create a conflicting version of hello.xml directly on
the master track
# | * 8888888 (GitWork) Initialize hello.xml inside the alternative GitWork context
# |/
# * e5f6g7h Post-merge structural snapshot
```

Step 4.4: Trigger the Textual Collision and Inspect Layout Markers

Attempt a standard automated merge reintegration procedure. This will intentionally fail, and Git will inject visual tracking collision markers into the structure:

```
$ git merge GitWork
# Auto-merging hello.xml
# CONFLICT (content): Merge conflict in hello.xml
# Automatic merge failed; fix conflicts and then commit the result.

$ cat hello.xml
<root><data>
<<<<<<< HEAD
Hello from the conflicting Master Trunk path
=======
Hello from the isolated GitWork branch path
>>>>>>> GitWork
</data></root>
```

Step 4.5: Engage 3-Way Structural Merge Resolution Engine

Launch the automated `git mergetool` orchestration driver to bring up the visual 3-way merge window layout in `P4Merge` (displaying the Base, Local, Remote, and final Result buffers simultaneously). Select the definitive correct data layout, save, and exit:

```
$ git mergetool
# Normal Merge conflict resolution wizard initiating...
# Visualizing comparison blocks... Resolution confirmed. File updated.
```

Step 4.6: Conclude Commit Lifecycle and Manage Artifact Remnants

Confirm status tracking changes. Note that a structural backup file named `hello.xml.orig` was written by the engine workspace automatically. Capture the clean merge resolution state, add the tracking extension block rules matrix inside `.gitignore`, and finalize the cleanup procedure:

```
$ git status
# Both paths modified: hello.xml
# Untracked files: hello.xml.orig

$ git add hello.xml
$ git commit -m "Resolve explicit multi-axis merge text collision within hello.xml
manually via P4Merge"

# Append tracking exclusion rule matrix for tool residue files:
$ echo "*.orig" >> .gitignore
$ git add .gitignore
$ git commit -m "Update .gitignore definitions to disregard external 3-way tool
resolution backup files (.orig)"

$ git branch -d GitWork
# Purge now-obsolete tracked reference label vector labels safely
$ git log --oneline --graph --decorate
```

Lab 5: Remote Cloud Sync Operations and Ecosystem Cleanup

Objectives: Maintain total workflow synchronization with centralized upstream remote servers by managing operational pull patterns, uploading finalized history nodes, and performing clean workspace verification.

Step 5.1: Verify Local Integrity and Environment Balance

Confirm that the workspace is fully synchronized locally and has no loose modifications pending attention:

```
$ git status
# On branch master
# nothing to commit, working tree clean

$ git branch -a
# * master
#   remotes/origin/master
```

Step 5.2: Ingest Upstream Remote Synchronization Differentials

Execute the data extraction update framework to query, retrieve, and weave downstream incoming changes from your centralized team repository securely:

```
$ git pull origin master
# From https://gitlab.com/your-username/GitDemo
#  * branch            master     -> FETCH_HEAD
# Already up to date.
```

Step 5.3: Propagate Localized Incremental History Nodes Vertically

Transmit the complete verified structured timeline block progression generated across the previous conflict resolution labs up to the active multi-user production system server stream:

```
$ git push origin master
# Counting objects: 12, done.
# Delta compression using up to 8 threads.
# Compressing objects: 100% (10/10), done.
# Writing objects: 100% (12/12), 1.25 KiB | 645.00 KiB/s, done.
# Total 12 (delta 4), reused 0 (delta 0)
# To https://gitlab.com/your-username/GitDemo.git
#    e5f6g7h..z9y8x7w  master -> master
```

Step 5.4: Structural State Analysis Verification

Perform a final diagnostic health check to confirm that your local environment matches the remote cluster baseline seamlessly:

```
$ git status
# On branch master
# Your branch is up to date with 'origin/master'.
# nothing to commit, working tree clean
```

> **Lab Completion Verification Checklist:** All 5 Git Hands-On Lab modules are now fully realized and operational. Local workspace structures, global runtime configuration profiles, file ignoring rules, branching pipelines, explicit 3-way multi-axis collision mitigation paths, and remote upstream pushing pipelines have been verified.